

Auditoria Enterprise SaaS

COMUNIC@TE

Principal Software Architect / Security Engineer / Senior SaaS Auditor

Fecha: 20 de mayo de 2026

Modo: read-only analysis. No se modifico codigo de la aplicacion.

Puntuacion general: 52/100. Madurez: MVP operativo / SaaS interno, no enterprise-ready. La base tecnica funciona, pero existen riesgos importantes en autorizacion, integridad de negocio, escalabilidad, compliance y mantenibilidad.

Verificaciones ejecutadas: npm run lint paso con 10 warnings; npm test fallo porque no existe script test; npm audit reporto 8 vulnerabilidades low transitivas; no se ejecuto build para mantener el modo read-only.

A. Resumen ejecutivo

- Estado general: aplicacion Vite/React 19 con Firebase Auth, Firestore y Netlify Functions. Backend canonico en netlify/functions.
- Fortalezas: validacion Zod server-side, reglas Firestore restrictivas para colecciones principales, autenticacion Google, consentimiento legal, paginacion inicial y funciones para operaciones criticas.
- Debilidades principales: ausencia de RBAC/multitenancy, integridad de IMEI no blindada en servidor, datos parciales por paginacion, falta de tests, compliance legal incompleto y deuda de frontend.
- Riesgo global: alto si se expone a mas usuarios, clientes o tiendas; medio si se mantiene como herramienta interna de pocos usuarios controlados.

Scores por area

- Seguridad: 5.5/10
- Arquitectura: 5/10
- Performance: 5/10
- UX/UI: 6/10
- Mantenibilidad: 4.5/10
- Escalabilidad: 4/10
- Calidad enterprise: 4.5/10

B. Hallazgos y soluciones

CRITICO C-01

No existe multitenancy ni RBAC real

Archivo(s): netlify/functions/registros.mjs:6, netlify/functions/ventas.mjs:6, firestore.rules:198, src/config/auth.js:1

Explicacion: Todos los usuarios autorizados operan sobre el mismo espacio users/shared. No hay separacion por empresa, tienda, rol o permiso granular.

Riesgo/impacto: Exposicion total entre usuarios autorizados, imposibilidad de auditoria enterprise por tenant, riesgo alto ante cuenta comprometida.

Solucion recomendada: Crear modelo organizations/{orgId}/users/{uid}/roles, claims custom en Firebase, checks server-side por accion y reglas Firestore alineadas por tenant.

Prioridad: P0 antes de crecer a mas usuarios o clientes.

CRITICO C-02

Unicidad de IMEI no garantizada en servidor

Archivo(s): netlify/functions/registros.mjs:28, netlify/functions/ventas.mjs:28, src/features/registros/RegistroForm.jsx:25

Explicacion: La prevencion de duplicados depende de datos cargados en frontend y puede fallar con paginacion, concurrencia o llamadas API directas.

Riesgo/impacto: Registros o ventas duplicadas del mismo equipo, inconsistencias contables y operativas.

Solucion recomendada: Implementar locks transaccionales por IMEI: imeiLocks/{imei} con estado registrado/vendido, validado dentro de la misma transaccion. Agregar tests de concurrencia.

Prioridad: P0 para integridad del dominio.

ALTO A-01

Búsqueda, clientes y KPIs operan con datos parciales

Archivo(s): src/app/App.jsx:31, src/app/App.jsx:214, src/features/clientes/ClientesList.jsx:35

Explicacion: La app carga 40 registros/ventas por pagina, pero clientes, busqueda global, historial e ingresos se calculan sobre lo cargado.

Riesgo/impacto: Decisiones con informacion incompleta, clientes invisibles, validaciones incorrectas.

Solucion recomendada: Mover busqueda, clientes operativos, totales y reportes a endpoints server-side con queries/indexes adecuados y paginacion por cursor.

Prioridad: P1.

ALTO A-02

Suscripciones realtime mezclan snapshot actual con estado previo

Archivo(s): src/app/App.jsx:221, src/app/App.jsx:246

Explicacion: setRegistros(prev => mergeByDateDesc(data, prev)) y setVentas equivalente retienen elementos anteriores aunque ya no pertenezcan al snapshot actual.

Riesgo/impacto: Datos borrados o fuera de pagina pueden seguir visibles hasta recarga o manipulacion local.

Solucion recomendada: Separar primera pagina realtime del cache de paginas adicionales; reconciliar docChanges added/modified/removed por id.

Prioridad: P1.

ALTO A-03

Autenticacion server-side no usa verificacion Admin robusta

Archivo(s): netlify/functions/_shared.mjs:67

Explicacion: Se valida ID token usando Identity Toolkit REST y una API key con fallback hard-coded.

Riesgo/impacto: No hay validacion centralizada de tokens revocados, usuarios deshabilitados, claims de rol o tenant.

Solucion recomendada: Usar firebase-admin auth().verifyIdToken(token, true), cargar permisos por custom claims y rechazar usuarios sin tenant/rol.

Prioridad: P1.

ALTO A-04

Rate limiting no distribuido

Archivo(s): netlify/functions/_shared.mjs:16, netlify/functions/_shared.mjs:87

Explicacion: El rate limit vive en un Map de memoria por instancia y por UID.

Riesgo/impacto: Se resetea por cold starts/instancias y no protege bien contra abuso distribuido ni consumo de RENIEC/Gemini.

Solucion recomendada: Implementar rate limit persistente por IP+UID+endpoint en Redis/Upstash/Firestore TTL, con limite previo a llamadas caras.

Prioridad: P1.

ALTO A-05

Headers de seguridad incompletos

Archivo(s): public/_headers:2, firebase.json:17

Explicacion: Existen nosniff, referrer, permissions y frame deny, pero falta CSP, HSTS, COOP/CORP y cache policy fuerte para assets.

Riesgo/impacto: Mayor impacto ante XSS o inyecciones de dependencias; proteccion incompleta en navegadores modernos.

Solucion recomendada: Definir CSP restrictiva por entorno, script-src con self y dominios necesarios, connect-src Firebase/Netlify/Gemini no directo, HSTS en hosting HTTPS y cache immutable para assets hash.

Prioridad: P1.

ALTO A-06

PII enviada a servicios externos sin controles enterprise suficientes

Archivo(s): netlify/functions/reniec.mjs:39, netlify/functions/analizarCajaGemini.mjs:31

Explicacion: DNI y fotos de cajas se envian a RENIEC/Codart y Gemini sin timeout, circuit breaker, auditoria de proveedor ni politica de retencion explicita.

Riesgo/impacto: Riesgo legal, privacidad, costos y disponibilidad.

Solucion recomendada: Agregar AbortController con timeout, tama?o maximo server-side, redaccion de logs, DPA/terminos proveedor, registro de finalidad y politica de retencion.

Prioridad: P1.

ALTO A-07

Compliance legal con placeholders y texto plantilla

Archivo(s): src/config/legal.js:16, src/features/legal/LegalDocumentPage.jsx:118

Explicacion: Pais, jurisdiccion y dominio siguen como placeholders; la propia UI declara que es plantilla operativa.

Riesgo/impacto: Documentos no defendibles como politica final, riesgo de incumplimiento local.

Solucion recomendada: Completar entidad legal, jurisdiccion, dominio, domicilio, derechos ARCO/GDPR aplicables, base legal, encargados, transferencias, retencion y contacto formal.

Prioridad: P1 antes de publicacion formal.

MEDIO M-01

Bundle inicial pesado y precarga de jsPDF

Archivo(s): dist/index.html, src/utls/pdfLibraries.js:1, vite.config.js:23

Explicacion: El dist actual precarga vendor-jspdf al inicio. Paquetes pesados: Firebase ~333 KB, jsPDF ~331 KB, html2canvas ~196 KB, React ~200 KB sin gzip.

Riesgo/impacto: Carga inicial lenta, peor experiencia movil y consumo innecesario.

Solucion recomendada: Convertir jsPDF/JsBarcode/html2canvas a imports dinamicos dentro de acciones PDF; ajustar manualChunks para evitar modulepreload de features no usadas.

Prioridad: P2.

MEDIO M-02

Componentes gigantes y responsabilidades mezcladas

Archivo(s): src/features/registros/RegistroForm.jsx, src/app/App.jsx, src/features/boletas/boletaPdf.js

Explicacion: Archivos de 500-630 lineas mezclan UI, validacion, dominio, datos y side effects.

Riesgo/impacto: Mantenibilidad baja, regresiones faciles, onboarding lento.

Solucion recomendada: Extraer hooks por feature, servicios de dominio, mappers payload, componentes pequenos y tests por modulo.

Prioridad: P2.

MEDIO M-03

No existe suite de tests aunque AGENTS.md exige npm test

Archivo(s): package.json:7, AGENTS.md:4

Explicacion: npm test falla por script inexistente.

Riesgo/impacto: Cambios de negocio sin red de seguridad.

Solucion recomendada: Agregar Vitest para utls/domain y pruebas de Netlify Functions con mocks de Firestore/Auth.

Incluir test en CI.

Prioridad: P1/P2.

MEDIO M-04

Warnings de React Hooks

Archivo(s): src/features/registros/RegistroForm.jsx, src/features/ventas/VentaForm.jsx, src/features/boletas/BoletaExtranjera.jsx

Explicacion: 10 warnings: setState sincronico en effects y dependencias faltantes.

Riesgo/impacto: Renders cascada, closures obsoletos, bugs intermitentes.

Solucion recomendada: Mover derivaciones a useMemo/useReducer, estabilizar callbacks con useCallback y corregir dependencias.

Prioridad: P2.

MEDIO M-05

Sin TypeScript ni contratos tipados

Archivo(s): src/**/*.jsx, netlify/functions/*.mjs

Explicacion: El dominio depende de strings y shapes implicitos.

Riesgo/impacto: Errores silenciosos en payloads, campos opcionales y migraciones.

Solucion recomendada: Migracion gradual: tipos JSDoc o TypeScript en domain/validators primero; inferir tipos desde Zod.

Prioridad: P2.

MEDIO M-06

Accesibilidad incompleta en modales y controles táctiles

Archivo(s): src/components/ui/ConfirmModal.jsx:23, src/index.css:208

Explicación: Modales sin role dialog, aria-modal, focus trap, Escape ni retorno de foco. Botones icono ~33px, debajo de 44px recomendado.

Riesgo/impacto: Incumplimiento WCAG y mala experiencia teclado/movil.

Solución recomendada: Crear componente Dialog accesible reutilizable, focus management, aria-labelledby/describedby, Escape, touch targets >=44px.

Prioridad: P2.

MEDIO M-07

Lint oculta imports muertos

Archivo(s): eslint.config.js:29, multiples archivos lucide-react

Explicación: varIgnorePattern ^[A-Z_] permite imports de iconos no usados; varios archivos importan casi todo lucide-react.

Riesgo/impacto: Ruido, bundle menos predecible y deuda técnica.

Solución recomendada: Eliminar la excepción amplia, importar solo iconos usados y evitar eslint-disable no-unused-vars.

Prioridad: P2.

MEDIO M-08

Logo se almacena como base64 en Firestore

Archivo(s): src/features/settings/ConfiguracionLogo.jsx:13, firestore.rules:165

Explicación: Imagen hasta 650 KB cliente / 900 KB reglas se guarda como documento.

Riesgo/impacto: Costos, latencia y documentos grandes para una imagen.

Solución recomendada: Mover logos a Firebase Storage o Cloudinary con metadata, validación MIME y reglas de acceso.

Prioridad: P2.

MEDIO M-09

Boletas escriben directo desde cliente

Archivo(s): src/features/boletas/BoletaExtranjera.jsx:95, firestore.rules:229

Explicación: La generación de boletas y contador se hace en frontend con reglas Firestore.

Riesgo/impacto: Menor control auditado y validaciones de negocio incompletas.

Solución recomendada: Mover emisión/reimpresión a Netlify Function con validación server-side, logs y control de correlativo.

Prioridad: P2.

MEDIO M-10

Consultas y joins O(n) en cliente

Archivo(s): src/features/clientes/CientesList.jsx:35, src/app/App.jsx:380

Explicación: Se hacen finds/filter repetidos sobre arrays de clientes, registros y ventas.

Riesgo/impacto: Degradación con miles de documentos.

Solución recomendada: Usar mapas memoizados por DNI/IMEI o consultas server-side especializadas.

Prioridad: P2.

BAJO B-01

Residuos de Vite/React starter

Archivo(s): src/App.css, src/assets/react.svg, public/vite.svg, index.html:7

Explicación: Branding inconsistente y archivos innecesarios.

Riesgo/impacto: Bajo, pero afecta percepción profesional.

Solución recomendada: Eliminar assets no usados y usar favicon/product icons propios.

Prioridad: P3.

BAJO B-02

Idioma HTML incorrecto

Archivo(s): index.html:2

Explicacion: lang=en en una app principalmente en espanol.

Riesgo/impacto: SEO/accesibilidad menores.

Solucion recomendada: Cambiar a lang=es-PE o es.

Prioridad: P3.

BAJO B-03

SEO limitado para SPA privada

Archivo(s): index.html, public/sitemap-legal.xml, src/utils/seo.js

Explicacion: Solo paginas legales ajustan metadata; no existe robots.txt visible.

Riesgo/impacto: Indexacion incompleta o accidental segun hosting.

Solucion recomendada: Agregar robots.txt, sitemap principal si aplica, noindex en rutas privadas y canonical estable.

Prioridad: P3.

BAJO B-04

Archivos/directorios obsoletos o confusos

Archivo(s): backend/, functions/, deno.lock, .agents/

Explicacion: README dice que Netlify Functions es el backend canonico, pero existen restos o herramientas versionadas.

Riesgo/impacto: Confusion operativa y superficie de mantenimiento.

Solucion recomendada: Documentar o limpiar en una tarea separada: backend vacio, functions legacy, deno.lock si no se usa.

Prioridad: P3.

BAJO B-05

Observabilidad insuficiente

Archivo(s): src/app/App.jsx, netlify/functions/*.mjs, scripts/local-api.mjs

Explicacion: Uso de console.error/warn sin logger estructurado, trazas, alertas ni correlacion de request.

Riesgo/impacto: Debug lento en produccion.

Solucion recomendada: Agregar logger estructurado en functions, requestId, eventos de auditoria por accion critica y monitoring.

Prioridad: P2/P3.

BAJO B-06

Vulnerabilidades npm low transitivas

Archivo(s): package-lock.json, firebase-admin dependency tree

Explicacion: npm audit reporta 8 low severity por @tootallnate/once via google-cloud/firestore/storage.

Riesgo/impacto: Bajo inmediato, pero debe monitorearse.

Solucion recomendada: Actualizar firebase-admin/google deps cuando haya version compatible; evitar npm audit fix --force sin prueba porque sugiere downgrade breaking.

Prioridad: P3.

BAJO B-07

No hay PRODUCT.md ni DESIGN.md

Archivo(s): raiz del proyecto

Explicacion: No existe fuente formal para principios de producto/diseno.

Riesgo/impacto: Inconsistencia visual y decisiones UX ad hoc.

Solucion recomendada: Crear PRODUCT.md y DESIGN.md con usuarios, flujos, tono, tokens, componentes y criterios de accesibilidad.

Prioridad: P3.

C. Roadmap recomendado

Quick wins: 1 a 3 dias

- Agregar script npm test con Vitest aunque al inicio cubra solo utils y validators.
- Corregir los 10 warnings de React Hooks.
- Limpiar imports muertos de lucide-react y quitar excepciones amplias de ESLint.
- Cambiar lang a es-PE, favicon Vite por icono propio, robots.txt y metadata base.
- Completar placeholders legales: pais, jurisdiccion y dominio.
- Agregar CSP/HSTS/COOP/CORP y cache immutable para assets hash.

Prioridad 1: seguridad e integridad

- Diseñar tenant/org/roles/permissions y migrar paths Firestore fuera de users/shared.
- Mover unicidad de IMEI a servidor con locks transaccionales.
- Usar Firebase Admin verifyIdToken(token, true), claims y revocation checks.
- Rate limiting distribuido por IP, UID y endpoint.
- Timeouts, payload limits y auditoria para RENIEC/Gemini.

Prioridad 2: escalabilidad y arquitectura

- Crear endpoints server-side para busqueda global, clientes operativos, KPIs e historiales.
- Separar dominio: domain/registros, domain/ventas, repositories/firestore, api/functions.
- Pasar boletas a backend con control de correlativo y logs de auditoria.
- Optimizar PDF: dynamic imports reales y evitar modulepreload inicial de jsPDF.

Prioridad 3: calidad enterprise

- Migracion gradual a TypeScript o JSDoc estricto con tipos inferidos de Zod.
- Observabilidad: requestId, structured logs, errores monitoreados y alertas.
- Dialog accesible reusable con focus trap, Escape y aria-modal.
- PRODUCT.md y DESIGN.md para criterios UX, componentes y consistencia visual.

D. Recomendaciones de refactor

- No iniciar por una reescritura visual. Primero blindar dominio, autorizacion e integridad.
- Extraer de App.jsx: useAuthSession, useFirestoreCollections, useGlobalSearch, useBackupExport y route/view state.
- Extraer de RegistroForm y VentaForm: schemas compartidos, mappers de payload, validaciones de paso, hooks de RENIEC y scanner.
- Convertir Netlify Functions en capas: handler -> authz -> validation -> service -> repository.
- Agregar pruebas unitarias para luhn, documentos, validators Zod, calculos de ventas, locks IMEI y legal consent.
- Usar storage para assets binarios como logos, no Firestore base64.

E. Notas de alcance

- No se realizo pentest activo contra produccion ni pruebas destructivas.
- No se ejecuto npm run build para no modificar dist en modo read-only.
- No se auditaron credenciales reales; los archivos .env estan ignorados por git segun git check-ignore.
- No aplica Prisma/SQL en el estado actual: la base de datos efectiva es Firestore.